

PROJECT REPORT

UVM FIFO

Submitted by : Sama Ashraf Mabrouk

Submitted to: Eng. Kareem Waseem

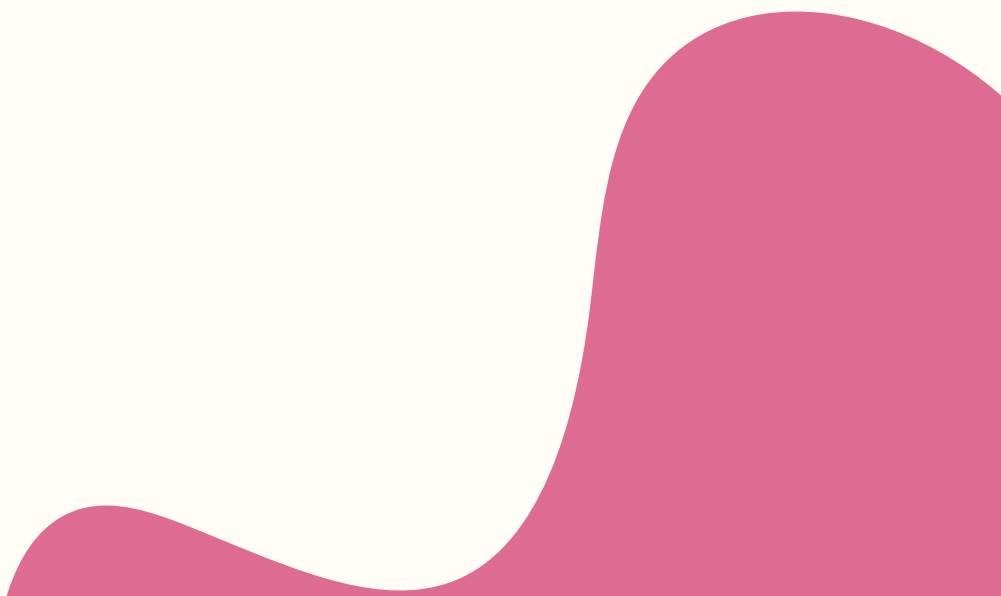
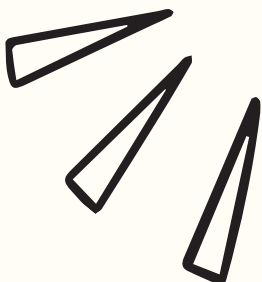


Table of Contents

1. Verification Plan	3
2. SVA.....	5
Assertions Tables.....	5
The Assertions Code:	6
3. UVM Structure.....	8
The Structure itself.....	8
How the UVM testbench work?	9
4. Code Coverage	11
5. Functional Coverage.....	13
6. Sequential Domain Coverage	13
7. Bug Report	14
8. Transcript	16
9. Questa-Sim snippets to for each UVM sequence	17
Sequence 1 :.....	17
Sequence 2 :.....	17
Sequence 3	18
Sequence 4	18
Sequence 5	19
10. Codes	20
1. Top Module.....	20
2. Interface.....	21
3. FIFO Test	21

4.	FIFO Environment	23
5.	FIFO Agent.....	23
6.	FIFO Monitor.....	24
7.	FIFO Driver	25
8.	Configuration Object.....	26
9.	Sequence Item	26
10.	FIFO Sequences Package	27
11.	FIFO Sequencer	29
12.	FIFO Scoreboard	30
13.	FIFO Coverage Collector.....	34
14.	Full Edited Design	35

Verification Plan

<i>Label Description</i>	<i>Stimulus Generation</i>	<i>Functional Coverage</i>	<i>Functionality check</i>
Sequence 1 Reset	Generates a reset condition by de-asserting all inputs (rst_n = 0, wr_en = 0, rd_en = 0)	Functional coverage is sampled to ensure reset behavior is exercised (rst_n = 0)	Verifies correct reset behavior, ensuring the FIFO is properly reset
Sequence 2 Write	Generates random write transactions, setting wr_en to 1, rd_en to 0, rst_n to 1, and randomizing data_in	Functional coverage samples different write data patterns and FIFO fill levels	Checks if write operations are correctly handled by the FIFO (data written, FIFO state updated)
Sequence 3 Read	Generates random read transactions, setting rd_en to 1, wr_en to 0, rst_n to 1, and randomizing data_in	Functional coverage ensures different read patterns and FIFO states are covered	Verifies the correct handling of read operations, ensuring the data is read

			correctly from the FIFO
Sequence 4 Read after Write	Generates a sequence with a mixture of write and read transactions, randomizing all fields including data_in	Cross-coverage of write and read operations is collected to ensure combinations of operations are covered	Checks the overall FIFO functionality, ensuring data consistency when both write and read operations occur together
Sequence 5 Reset	Generates a reset condition by deasserting all inputs (rst_n = 0, wr_en = 0, rd_en = 0)	Functional coverage is sampled to ensure reset behavior is exercised (rst_n = 0)	Verifies correct reset behavior, ensuring the FIFO is properly reset



Assertions Tables

A	B
Feature	Assertion
When the FIFO is empty and count is 0, empty is asserted	EMPTY_assertion : assert (empty && !full && !almostempty && !almostfull)
When the FIFO is almost empty and count is 1, almostempty is asserted	ALMOSTEMPTY_assertion : assert (!empty && !full && almostempty && !almostfull)
When the FIFO is almost full and count is FIFO_DEPTH - 1, almostfull is asserted	ALMOSTFULL_assertion : assert (!empty && !full && !almostempty and almostfull)
When the FIFO is full and count is FIFO_DEPTH, full is asserted	FULL_assertion : assert (!empty && full && !almostempty && !almostfull)
Overflow condition when FIFO is full and wr_en is high	OVERFLOW_assertion : assert property (OVERFLOW_FIFO)
Underflow condition when FIFO is empty and rd_en is high	UNDERFLOW_assertion : assert property (UNDERFLOW_FIFO)
When wr_en is high and count < FIFO_DEPTH and not full, wr_ack is asserted	WR_ACK_HIGH_assertion : assert property (WR_ACK_HIGH)
When wr_en is high and FIFO is full, wr_ack is low	WR_ACK_LOW_assertion : assert property (WR_ACK_LOW)
The counter is reset to 0 when rst_n is low	COUNTER_0_assertion : assert property (COUNTER_0)
The counter increments by 1 when wr_en is high and rd_en is low and not full	COUNTER_INC_10_assertion : assert property (COUNTER_INC_10)
The counter decrements by 1 when rd_en is high and wr_en is low and not empty	COUNTER_INC_01_assertion : assert property (COUNTER_INC_01)
The counter increments by 1 when both wr_en and rd_en are high and FIFO is empty	COUNTER_INC_11_WR_assertion : assert property (COUNTER_INC_11_WR)
The counter decrements by 1 when both wr_en and rd_en are high and FIFO is full	COUNTER_INC_11_RD_assertion : assert property (COUNTER_INC_11_RD)
The counter remains unchanged when both wr_en and rd_en are high and FIFO is full/empty	COUNTER_LAT_assertion : assert property (COUNTER_LAT)
The rd_ptr is reset when rst_n is low	PTR_RST_assertion : assert property (PTR_RST)
The rd_ptr increments when rd_en is high and count is not 0	RD_PTR_assertion : assert property (RD_PTR)
The wr_ptr increments when wr_en is high and count < FIFO_DEPTH	WR_PTR_assertion : assert property (WR_PTR)

A	B	C	D
Assertion Name	Condition	Property Type	Message on Failure
EMPTY_assertion	count == 0	Combinational	EMPTY_assertion fail :{
ALMOSTEMPTY_assertion	count == 1	Combinational	ALMOSTEMPTY_assertion fail :{
ALMOSTFULL_assertion	count == FIFO_DEPTH-1	Combinational	ALMOSTFULL_assertion fail :{
FULL_assertion	count == FIFO_DEPTH	Combinational	FULL_assertion fail :{
OVERFLOW_assertion	full & wr_en	Overflow	OVERFLOW_assertion
UNDERFLOW_assertion	empty && rd_en	Underflow	UNDERFLOW_assertion
WR_ACK_HIGH_assertion	wr_en && (count < FIFO_DEPTH) && !full	wr_ack	WR_ACK_HIGH_assertion
WR_ACK_LOW_assertion	wr_en && full	wr_ack	WR_ACK_LOW_assertion
COUNTER_0_assertion	!rst_n	Counter	COUNTER_0_assertion
COUNTER_INC_10_assertion	((wr_en, rd_en) == 2'b10) && !full	Counter	COUNTER_INC_WR_assertion fail
COUNTER_INC_01_assertion	((wr_en, rd_en) == 2'b01) && !empty	Counter	COUNTER_INC_WR_assertion fail
COUNTER_INC_11_WR_assertion	((wr_en, rd_en) == 2'b11) && empty	Counter	COUNTER_INC_WR_assertion fail
COUNTER_INC_11_RD_assertion	((wr_en, rd_en) == 2'b11) && full	Counter	COUNTER_INC_WR_assertion fail
COUNTER_LAT_assertion	((wr_en, rd_en) == 2'b01) && empty) (((wr_en, rd_en) == 2'b10) && full)	Counter	COUNTER_LAT_assertion fail
PTR_RST_assertion	!rst_n	Pointer	PTR_RST_assertion fail
RD_PTR_assertion	rd_en && (count != 0)	Pointer	RD_PTR_assertion fail
WR_PTR_assertion	wr_en && (count < FIFO_DEPTH)	Pointer	WR_PTR_assertion fail

The Assertions Code:

```

1  import FIFO_shared_pkg::*;
2
3  module fifo_sva(FIFO_if.DUT fifo_if);
4
5      logic [FIFO_WIDTH-1:0] data_in;
6      logic clk, rst_n, wr_en, rd_en;
7      logic [FIFO_WIDTH-1:0] data_out;
8      logic wr_ack, overflow;
9      logic full, empty, almostfull, almostempty, underflow;
10     localparam max_fifo_addr = $clog2(FIFO_DEPTH);
11
12     assign clk          = fifo_if.clk;
13     assign data_in      = fifo_if.data_in;
14     assign rst_n        = fifo_if.rst_n;
15     assign wr_en        = fifo_if.wr_en;
16     assign rd_en        = fifo_if.rd_en;
17     assign data_out     = fifo_if.data_out;
18     assign wr_ack       = fifo_if.wr_ack;
19     assign overflow     = fifo_if.overflow;
20     assign full         = fifo_if.full;
21     assign empty        = fifo_if.empty;
22     assign almostfull   = fifo_if.almostfull;
23     assign almostempty  = fifo_if.almostempty;
24     assign underflow    = fifo_if.underflow;
25
26     //-----ASSERTIONS-----
27
28     `ifdef SIM
29
30         // Assertions for Combinational Outputs
31         always_comb begin
32             if (count == 0) begin
33                 EMPTY_assertion : assert (empty && !full && !almostempty && !almostfull) else $display("EMPTY_assertion fail : ( ");
34                 EMPTY_cover : cover (empty && !full && !almostempty && !almostfull) $display("EMPTY_assertion Pass :D");
35             end
36             if (count == 1) begin
37                 ALMOSTEMPTY_assertion : assert (!empty && !full && almostempty && !almostfull) else $display("ALMOSTFULL_assertion fail : ( ");
38                 ALMOSTEMPTY_cover : cover (!empty && !full && almostempty && !almostfull) $display("ALMOSTFULL_assertion Pass :D");
39             end
40             if (count == FIFO_DEPTH-1) begin
41                 ALMOSTFULL_assertion : assert (!empty && !full && !almostempty && almostfull) else $display("ALMOSTFULL_assertion fail : ( ");
42                 ALMOSTFULL_cover : cover (!empty && !full && !almostempty && almostfull) $display("ALMOSTEMPTY_assertion Pass :D");
43             end
44             if (count == FIFO_DEPTH) begin
45                 FULL_assertion : assert (!empty && full && !almostempty && !almostfull) else $display("FULL_assertion fail : ( ");
46                 FULL_cover : cover (!empty && full && !almostempty && !almostfull) $display("FULL_assertion Pass :D");
47             end
48         end
49     `endif

```

```

50     // Assertions for Overflow and Underflow
51     property OVERFLOW_FIFO;
52     @(posedge clk) disable iff (!rst_n) (full && wr_en) |>=> (overflow);
53     endproperty
54
55     property UNDERFLOW_FIFO;
56     @(posedge clk) disable iff (!rst_n) (empty && rd_en) |>=> (underflow);
57     endproperty
58
59     // Assertions for wr_ack
60     property WR_ACK_HIGH;
61     @(posedge clk) disable iff (!rst_n) (wr_en && (count < FIFO_DEPTH) && !full) |>=> (wr_ack);
62     endproperty
63
64     property WR_ACK_LOW;
65     @(posedge clk) disable iff (!rst_n) (wr_en && full) |>=> (!wr_ack);
66     endproperty
67
68     // Assertions for The Counter
69     property COUNT_0;
70     @(posedge clk) (!rst_n) |>=> (count == 0);
71     endproperty
72
73     property COUNT_INC_10;
74     @(posedge clk) disable iff (!rst_n) (((wr_en, rd_en) == 2'b10) && !full) |>=> (count == $past(count) + 1);
75     endproperty
76
77     property COUNT_INC_01;
78     @(posedge clk) disable iff (!rst_n) (((wr_en, rd_en) == 2'b01) && !empty) |>=> (count == $past(count) - 1);
79     endproperty
80
81     property COUNT_INC_11_WR;
82     @(posedge clk) disable iff (!rst_n) (((wr_en, rd_en) == 2'b11) && empty) |>=> (count == $past(count) + 1);
83     endproperty
84
85     property COUNT_INC_11_RD;
86     @(posedge clk) disable iff (!rst_n) (((wr_en, rd_en) == 2'b11) && full) |>=> (count == $past(count) - 1);
87     endproperty
88
89     property COUNT_LAT;
90     @(posedge clk) disable iff (!rst_n) (((wr_en, rd_en) == 2'b01) && empty) || (((wr_en, rd_en) == 2'b10) && full) |>=> (count == $past(count));
91     endproperty
92

```

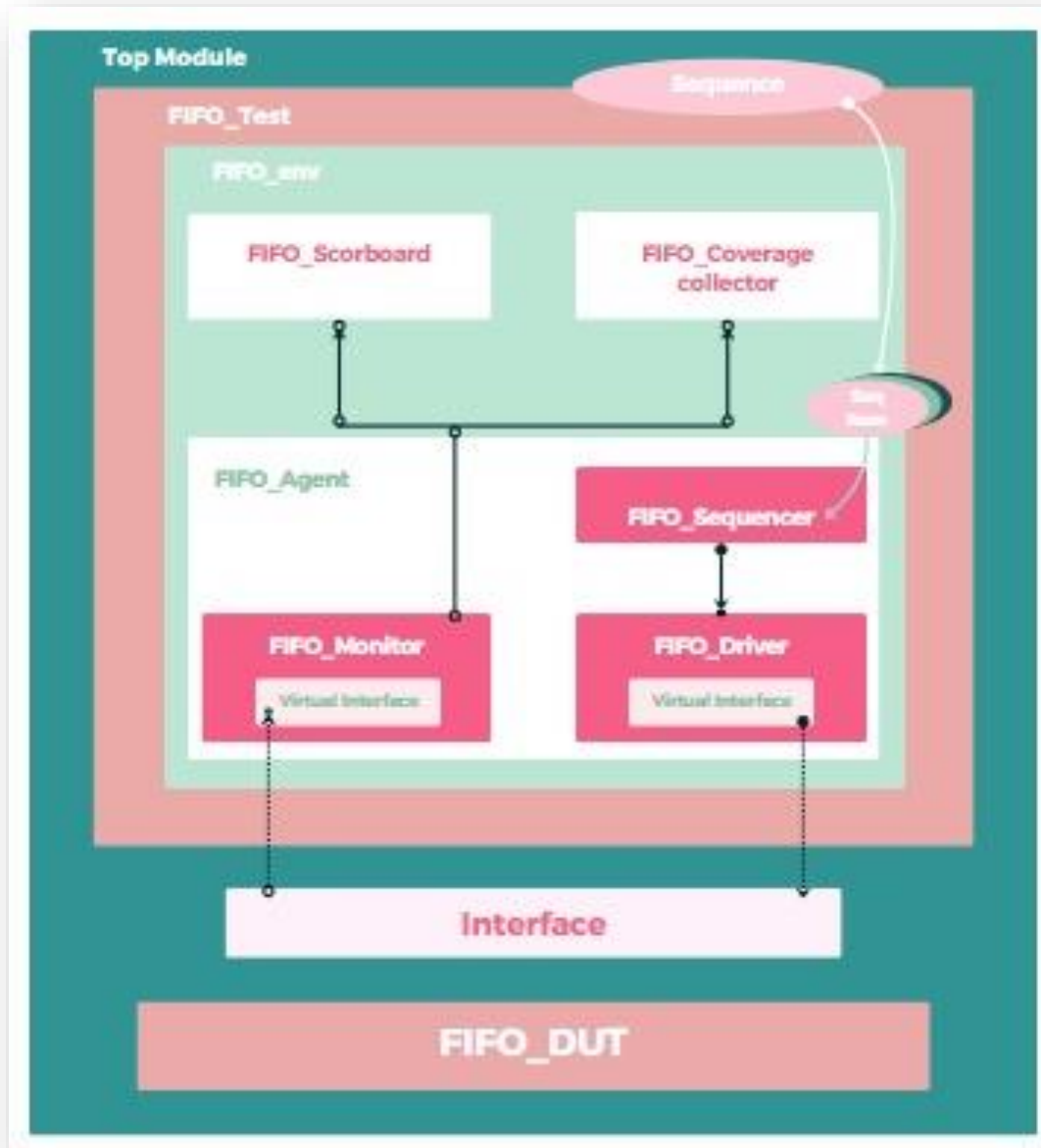
```

92
93 // Assertions for Pointers
94 property PTR_RST;
95   @(posedge clk) (!rst_n) |>=> (~rd_ptr && ~wr_ptr);
96 endproperty
97
98 property RD_PTR;
99   @(posedge clk) disable iff (!rst_n) (rd_en && (count != 0)) |>=> (rd_ptr == ($past(rd_ptr) + 1) % FIFO_DEPTH);
100 endproperty
101
102 property WR_PTR;
103   @(posedge clk) disable iff (!rst_n) (wr_en && (count < FIFO_DEPTH)) |>=> (wr_ptr == ($past(wr_ptr) + 1) % FIFO_DEPTH);
104 endproperty
105
106 // Assert Properties
107 OVERFLOW_assertion      : assert property (OVERFLOW_FIFO)      else $display("OVERFLOW_assertion");
108 UNDERFLOW_assertion    : assert property (UNDERFLOW_FIFO)    else $display("UNDERFLOW_assertion");
109 WR_ACK_HIGH_assertion   : assert property (WR_ACK_HIGH)       else $display("WR_ACK_HIGH_assertion");
110 WR_ACK_LOW_assertion    : assert property (WR_ACK_LOW)        else $display("WR_ACK_LOW_assertion");
111 COUNTER_0_assertion     : assert property (COUNT_0)          else $display("COUNTER_0_assertion");
112 COUNTER_INC_10_assertion : assert property (COUNT_INC_10)    else $display("COUNTER_INC_WR_assertion fail");
113 COUNTER_INC_01_assertion : assert property (COUNT_INC_01)    else $display("COUNTER_INC_WR_assertion fail");
114 COUNTER_INC_11_WR_assertion : assert property (COUNT_INC_11_WR) else $display("COUNTER_INC_WR_assertion fail");
115 COUNTER_INC_11_RD_assertion : assert property (COUNT_INC_11_RD) else $display("COUNTER_INC_WR_assertion fail");
116 COUNTER_LAT_assertion   : assert property (COUNT_LAT)        else $display("COUNTER_LAT_assertion fail");
117 PTR_RST_assertion       : assert property (PTR_RST)           else $display("PTR_RST_assertion fail");
118 RD_PTR_assertion        : assert property (RD_PTR)            else $display("RD_PTR_assertion fail");
119 WR_PTR_assertion        : assert property (WR_PTR)            else $display("WR_PTR_assertion fail");
120
121 // Cover Properties
122 OVERFLOW_cover          : cover property (OVERFLOW_FIFO)      $display("OVERFLOW_assertion");
123 UNDERFLOW_cover        : cover property (UNDERFLOW_FIFO)    $display("UNDERFLOW_assertion");
124 WR_ACK_HIGH_cover       : cover property (WR_ACK_HIGH)       $display("WR_ACK_HIGH_assertion");
125 WR_ACK_LOW_cover        : cover property (WR_ACK_LOW)        $display("WR_ACK_LOW_assertion");
126 COUNTER_0_cover         : cover property (COUNT_0)          $display("COUNTER_0_assertion");
127 COUNTER_INC_10_cover     : cover property (COUNT_INC_10)    $display("COUNTER_INC_WR_assertion Pass");
128 COUNTER_INC_01_cover     : cover property (COUNT_INC_01)    $display("COUNTER_INC_WR_assertion Pass");
129 COUNTER_INC_11_WR_cover  : cover property (COUNT_INC_11_WR) $display("COUNTER_INC_WR_assertion Pass");
130 COUNTER_INC_11_RD_cover  : cover property (COUNT_INC_11_RD) $display("COUNTER_INC_WR_assertion Pass");
131 COUNTER_LAT_cover        : cover property (COUNT_LAT)        $display("COUNTER_LAT_assertion Pass");
132 PTR_RST_cover           : cover property (PTR_RST)           $display("PTR_RST_assertion pass");
133 RD_PTR_cover            : cover property (RD_PTR)            $display("RD_PTR_assertion pass");
134 WR_PTR_cover            : cover property (WR_PTR)            $display("WR_PTR_assertion pass");
135
136 `endif
137
138 endmodule

```

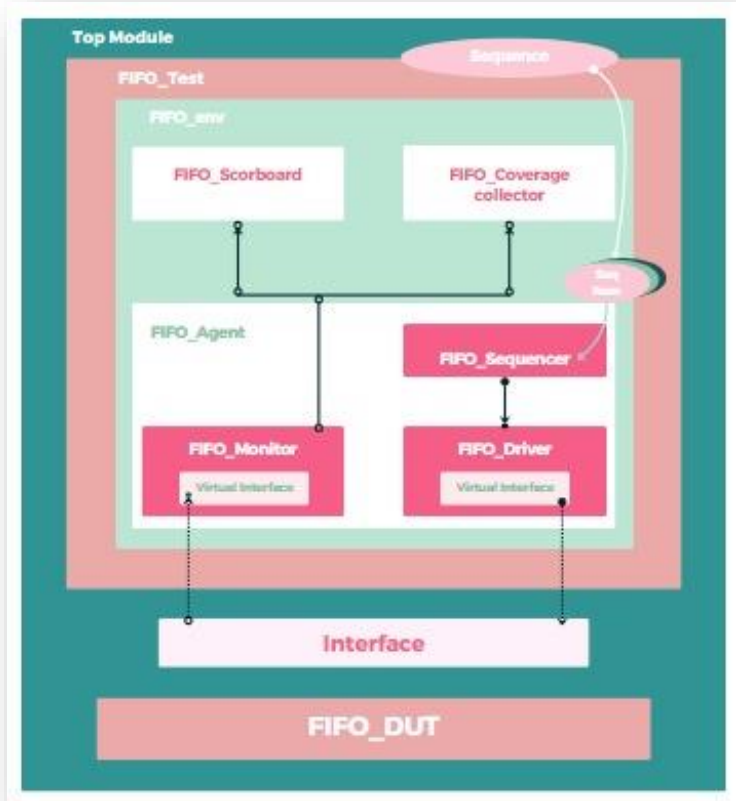

UVM Structure

The Structure itself



How the UVM testbench work?

- Starting from the **top module**, where clock generation, interface instantiation, assertion binding, and UVM test initiation are carried out, the design progresses toward the **FIFO_Test** via the Configuration Database in the UVM structure.



- In the **FIFO_Test**, the FIFO environment is constructed. The virtual interface is retrieved from the Configuration Database, and the configuration object is set back into the Configuration Database, allowing the agent to retrieve it during the run phase. During this phase, sequences drive transactions into the sequencer, which are then retrieved by the driver and applied to the **DUT** through the interface. My verification plan includes four sequences: **Reset_Sequence**, **Write_Only_Sequence**, **Read_Only_Sequence**, and **Write_Read_Sequence**. In the latter, read and write enable signals (rd_en and wr_en) are randomized under specific constraints, and these sequences are triggered by the test.

- The sequencer forwards these transactions to the driver, which applies them to the DUT via the virtual interface (fifo_if). The **FIFO monitor** captures the signal information from the virtual interface and translates it into sequence items, broadcasting them through the analysis port. These analysis ports are connected to the agent's analysis port, sending the sequence items to the **FIFO_Scoreboard** and the **FIFO_Coverage_Collector** through the same analysis ports.
- The **FIFO_Scoreboard** receives these sequence items, processes the inputs through the reference model, compares the expected output with the received sequence items, and reports any errors or correct results. The **FIFO_Coverage_Collector**, on the other hand, samples the data fields from these sequence items for functional coverage.
- Within the **FIFO_Agent**, where the sequencer, monitor, and driver are built, the agent retrieves the configuration object and connects its analysis ports with those of the agent. The driver's port is connected to the sequencer's export. Finally, the **FIFO_Environment** builds the **FIFO_Agent**, Scoreboard, and Coverage Collector, connecting the analysis port with the Scoreboard export and Coverage Collector.

Code Coverage

Code Coverage Analysis

File Bookmarks Window Help

Code Coverage Analysis

Statements - by instance (FIFO_TOP/DUT)

Statement

01.FIFO.sv

```
26 assign clk = fifo_if.clk;
27 assign data_in = fifo_if.data_in;
28 assign rst_n = fifo_if.rst_n;
29 assign wr_en = fifo_if.wr_en;
30 assign rd_en = fifo_if.rd_en;
50 always @(posedge clk or negedge rst_n) begin
51 wr_ptr <= 0;
52 overflow <= 0;
53 wr_ack <= 0;
54 mem[wr_ptr] <= data_in;
55 wr_ptr <= wr_ptr + 1;
56 overflow <= 0;
57 wr_ack <= 0;
58 overflow <= 0;
59 overflow <= 0;
76 always @(posedge clk or negedge rst_n) begin
77 rd_ptr <= 0;
78 underflow <= 0; //-----> Bug 2: i added this line because reset resets underflow
82 data_out <= mem[rd_ptr];
83 rd_ptr <= rd_ptr + 1;
84 underflow <= 0;
85 underflow <= 0;
86 underflow <= 0;
101 always @(posedge clk or negedge rst_n) begin
102 count <= 0;
103 count <= count + 1;
104 count <= count - 1;
105 count <= count + 1;
106 count <= count - 1;
122 assign full = (count == FIFO_DEPTH) ? 1 : 0;
123 assign empty = (count == 0) ? 1 : 0;
124 assign almostfull = (count == FIFO_DEPTH-1) ? 1 : 0; // Bug 8: fifo depth -2 --> fifo depth -1
125 assign almostempty = (count == 1) ? 1 : 0;
```

11.FIFO_TOP.sv

Code Coverage Analysis

File Bookmarks Window Help

Code Coverage Analysis

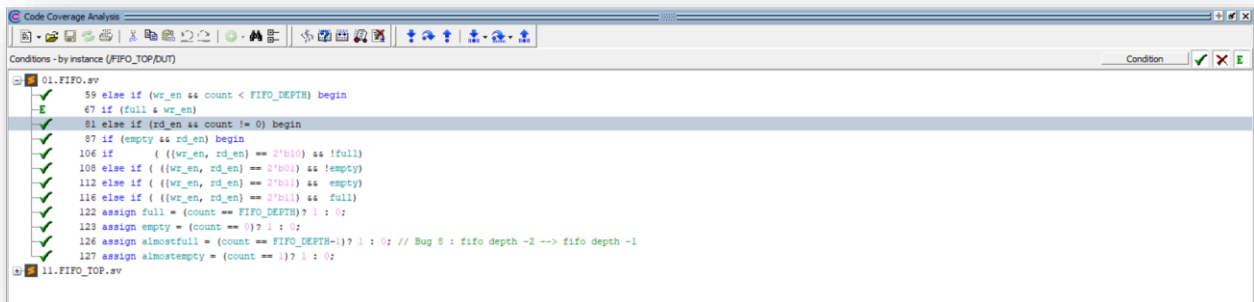
Branches - by instance (FIFO_TOP/DUT)

Branch

01.FIFO.sv

```
51 if (!rst_n) begin
52 else if (wr_en && count < FIFO_DEPTH) begin
53 else begin
54 if (full && wr_en)
55 else
56 if (!rst_n) begin
57 else if (rd_en && count != 0) begin
58 else begin //-----> Bug 3: this else branch i added to handle the underflow signal in a sequential manner (so i use non-blocking)
59 if (empty && rd_en) begin
60 else
61 if (!rst_n) begin
62 else begin
63 if ( ((wr_en, rd_en) == 2'b10) && !full)
64 else if ( ((wr_en, rd_en) == 2'b01) && !empty)
65 else if ( ((wr_en, rd_en) == 2'b11) && !empty)
66 else if ( ((wr_en, rd_en) == 2'b11) && full)
122 assign full = (count == FIFO_DEPTH) ? 1 : 0;
123 assign empty = (count == 0) ? 1 : 0;
124 assign almostfull = (count == FIFO_DEPTH-1) ? 1 : 0; // Bug 8: fifo depth -2 --> fifo depth -1
125 assign almostempty = (count == 1) ? 1 : 0;
```

11.FIFO_TOP.sv



1 It wasn't a bug but it's redundant condition, I commented it to achieve 100% conditional coverage



Functional Coverage

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/FIFO_coverage_p...		100.00%	100	100.00...					
+ TYPE cvr_g...		100.00%	100	100.00...				auto(1)	
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CVP cvr_g...		100.00%	100	100.00...					
+ CROSS cvr...		100.00%	100	100.00...					
+ CROSS cvr...		100.00%	100	100.00...					
+ CROSS cvr...		100.00%	100	100.00...					
+ CROSS cvr...		100.00%	100	100.00...					
+ CROSS cvr...		100.00%	100	100.00...					
+ CROSS cvr...		100.00%	100	100.00...					
+ CROSS cvr...		100.00%	100	100.00...					

Sequential Domain Coverage

Directive Coverage:

Directives 17 17 0 100.00%

Assertion Coverage:

Assertions 17 17 0 100.00%

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
▲ /FIFO_TOP/DUT/fifo_sva_inst/EMPTY_cover	SVA	✓	Off	940	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/ALMOSTEMPTY_cover	SVA	✓	Off	1065	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/ALMOSTFULL_cover	SVA	✓	Off	545	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/FULL_cover	SVA	✓	Off	369	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/OVERFLOW_cover	SVA	✓	Off	628	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/UNDERFLOW_cover	SVA	✓	Off	361	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/WR_ACK_HIGH_cover	SVA	✓	Off	5129	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/WR_ACK_LOW_cover	SVA	✓	Off	628	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/COUNTER_0_cover	SVA	✓	Off	979	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/COUNTER_INC_10_cover	SVA	✓	Off	3514	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/COUNTER_INC_01_cover	SVA	✓	Off	605	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/COUNTER_INC_11_WR_cover	SVA	✓	Off	257	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/COUNTER_INC_11_RD_cover	SVA	✓	Off	183	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/COUNTER_LAT_cover	SVA	✓	Off	549	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/PTR_RST_cover	SVA	✓	Off	979	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/RD_PTR_cover	SVA	✓	Off	2146	1	Unli...	1	100%		✓	0	0	0 ns	
▲ /FIFO_TOP/DUT/fifo_sva_inst/WR_PTR_cover	SVA	✓	Off	5129	1	Unli...	1	100%		✓	0	0	0 ns	

Bug Report

1. Re-setting Overflow, wr_Ack, underflow

```
47 //-----Write Operation-----
48
49
50 always @(posedge clk or negedge rst_n) begin
51     if (!rst_n) begin
52         wr_ptr <= 0;
53
54         //-----> Bug 1 & 1: I added those lines for overflow & wr_ack bec reset resets all the signals related to write operation :D
55         overflow <= 0; //<---
56         wr_ack <= 0; //<---
57     end
58 end
```

2. Bug 2: I added this line because reset resets underflow

```
71     end
72 end
73
74 //-----Read Operation-----
75
76 always @(posedge clk or negedge rst_n) begin
77     if (!rst_n) begin
78         rd_ptr <= 0;
79         underflow <= 0; //-----> Bug 2: i added this line because reset resets underflow
80     end
81     else if (rd_en && count != 0) begin
82         data_out <= mem[rd_ptr];
83         rd_ptr <= rd_ptr + 1;
84         underflow <= 0;
```

3. Bug 3: If both the read and write enable signals are high and the FIFO is empty, only writing will occur, and conversely, if the FIFO is full, only reading will take place. While this scenario was accounted for in the design, the counter handling was not implemented for these conditions.

```
84         underflow <= 0;
85     end
86     else begin //-----> Bug 3 : this else branch i added to handle the underflow signal in a sequential manner (so i use non-blocking)
87         if (empty && rd_en) begin
88             underflow <= 1;
89         end
90         else
91             underflow <= 0;
92     end
93 end
94
95 end
96
97
```

4. Bug 4: this else branch i added to handle the counter incrementation If a read and write enables were high and the FIFO was empty

```
99 //-----Counter -----
100
101 always @(posedge clk or negedge rst_n) begin
102     if (rst_n) begin
103         count <= 0;
104     end
105     else begin
106         if ( ({wr_en, rd_en} == 2'b10) && !full)
107             count <= count + 1;
108         else if ( ({wr_en, rd_en} == 2'b01) && !empty)
109             count <= count - 1;
110
111         //-----> Bug 4 : this else branch i added to handle the counter incrementation If a read and write enables were high and the FIFO was empty
112         else if ( ({wr_en, rd_en} == 2'b11) && empty)
113             count <= count + 1;
```

5. Bug 5: this else branch i added to handle the counter decrementation If a read and write enables were high and the the FIFO was full.

```
114
115 //-----> Bug 5 : this else branch i added to handle the counter decrementation If a read and write enables were high and the the FIFO was full.
116 else if ( ({wr_en, rd_en} == 2'b11) && full)
117     count <= count - 1;
118
119 end
120 end
```

6. Bug 6: underflow should be sequential,

should remove--> assign underflow = (empty && rd_en)? 1 : 0;

```
124
125 //-----> Bug 6: underflow should be sequential //---should removed--> assign underflow = (empty && rd_en)? 1 : 0;
126 assign almostfull = (count == FIFO_DEPTH-1)? 1 : 0; // Bug 7: fifo depth -2 --> fifo depth -1
```

7. Bug 7: As the almost full allow 1 other write operation NOT two

```
127 //-----> Bug 6: underflow should be sequential //---should removed--> assign underflow = (empty && rd_en)? 1 : 0;
126
127 assign almostfull = (count == FIFO_DEPTH-1)? 1 : 0; // Bug 7: fifo depth -2 --> fifo depth -1
128 assign almostempty = (count == 1)? 1 : 0;
129
```


Transcript

```
Transcript
# UVM_INFO verillog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verillog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init+struct
# UVM_INFO @ 0: reporter [RNTST] Running test fifo_test...
# UVM_INFO 10.FIFO_test_pkg.sv(51) @ 0: uvm_test_top [run_phase] Reset Asserted
# UVM_INFO 10.FIFO_test_pkg.sv(54) @ 20: uvm_test_top [run_phase] Reset Deasserted
# UVM_INFO 10.FIFO_test_pkg.sv(57) @ 20: uvm_test_top [run_phase] Stimulus Generation write sequence 2 Started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 20: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 20: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 40: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 60: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 80: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 100: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 120: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 140: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 160: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 180: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 200: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 220: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 240: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 260: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 280: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 300: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(35) @ 340: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is ENDED
# UVM_INFO 10.FIFO_test_pkg.sv(60) @ 340: uvm_test_top [run_phase] Stimulus Generation write sequence 2 Ended
# UVM_INFO 10.FIFO_test_pkg.sv(63) @ 340: uvm_test_top [run_phase] Stimulus Generation read sequence 3 Started
# UVM_INFO 3.reset_sequence_pkg.sv(52) @ 340: uvm_test_top.env.agt.sqr8brd_seq [body] 03 :: fifo_read_seq is started
# UVM_INFO 3.reset_sequence_pkg.sv(52) @ 340: uvm_test_top.env.agt.sqr8brd_seq [body] 03 :: fifo_read_seq is started
# UVM_INFO 10.FIFO_test_pkg.sv(66) @ 660: uvm_test_top [run_phase] Stimulus Generation read sequence 3 Ended
# UVM_INFO 10.FIFO_test_pkg.sv(69) @ 660: uvm_test_top [run_phase] Stimulus Generation read after write sequence 4 Started
# UVM_INFO 3.reset_sequence_pkg.sv(52) @ 660: uvm_test_top.env.agt.sqr8brd_seq [body] 04 :: fifo_write_read_seq is started
# UVM_INFO 3.reset_sequence_pkg.sv(59) @ 200660: uvm_test_top.env.agt.sqr8brd_seq [body] 04 :: fifo_write_read_seq is ENDED
```

```
Transcript
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 280: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 300: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 320: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(35) @ 340: uvm_test_top.env.agt.sqr8bwr_seq [body] 02 :: fifo_write_seq is ENDED
# UVM_INFO 10.FIFO_test_pkg.sv(60) @ 340: uvm_test_top [run_phase] Stimulus Generation write sequence 2 Ended
# UVM_INFO 10.FIFO_test_pkg.sv(63) @ 340: uvm_test_top [run_phase] Stimulus Generation read sequence 3 Started
# UVM_INFO 3.reset_sequence_pkg.sv(52) @ 340: uvm_test_top.env.agt.sqr8brd_seq [body] 03 :: fifo_read_seq is started
# UVM_INFO 3.reset_sequence_pkg.sv(52) @ 340: uvm_test_top.env.agt.sqr8brd_seq [body] 03 :: fifo_read_seq is ENDED
# UVM_INFO 10.FIFO_test_pkg.sv(66) @ 660: uvm_test_top [run_phase] Stimulus Generation read sequence 3 Ended
# UVM_INFO 10.FIFO_test_pkg.sv(69) @ 660: uvm_test_top [run_phase] Stimulus Generation read after write sequence 4 Started
# UVM_INFO 3.reset_sequence_pkg.sv(52) @ 660: uvm_test_top.env.agt.sqr8brd_seq [body] 04 :: fifo_write_read_seq is started
# UVM_INFO 3.reset_sequence_pkg.sv(59) @ 200660: uvm_test_top.env.agt.sqr8brd_seq [body] 04 :: fifo_write_read_seq is ENDED
# UVM_INFO 10.FIFO_test_pkg.sv(72) @ 200660: uvm_test_top [run_phase] Stimulus Generation read after write sequence 4 Ended
# UVM_INFO 10.FIFO_test_pkg.sv(75) @ 200660: uvm_test_top [run_phase] final :: Reset Deasserted
# UVM_INFO 10.FIFO_test_pkg.sv(78) @ 200660: uvm_test_top [run_phase] final :: Reset Deasserted
# UVM_INFO verillog_src/uvm-1.1d/src/base/uvm_object_base.sv(1247) @ 200660: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO 7.FIFO_scoreboard_pkg.sv(201) @ 200680: uvm_test_top.env.sb [report_phase] Total Successful transaction: 10034
# UVM_INFO 7.FIFO_scoreboard_pkg.sv(202) @ 200680: uvm_test_top.env.sb [report_phase] Total Failed transaction: 0

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 38
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0

** Report counts by id
[Questa UVM] 2
[RNTST] 1
[TEST_DONE] 1
[body] 22
[report_phase] 2
[run_phase] 10

** Note: $finish : D:/Eng.Kareem/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
Time: 200680 ns Iteration: 61 Instance: /FIFO_TOP

Project: SV Now: 200,680 ns Delta: 61 sim:/FIFO_TOP/#INITIAL#29 Coverage: Coverage: 100.00% Recursive Mode
```

```
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 38
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0

** Report counts by id
[Questa UVM] 2
[RNTST] 1
[TEST_DONE] 1
[body] 22
[report_phase] 2
[run_phase] 10

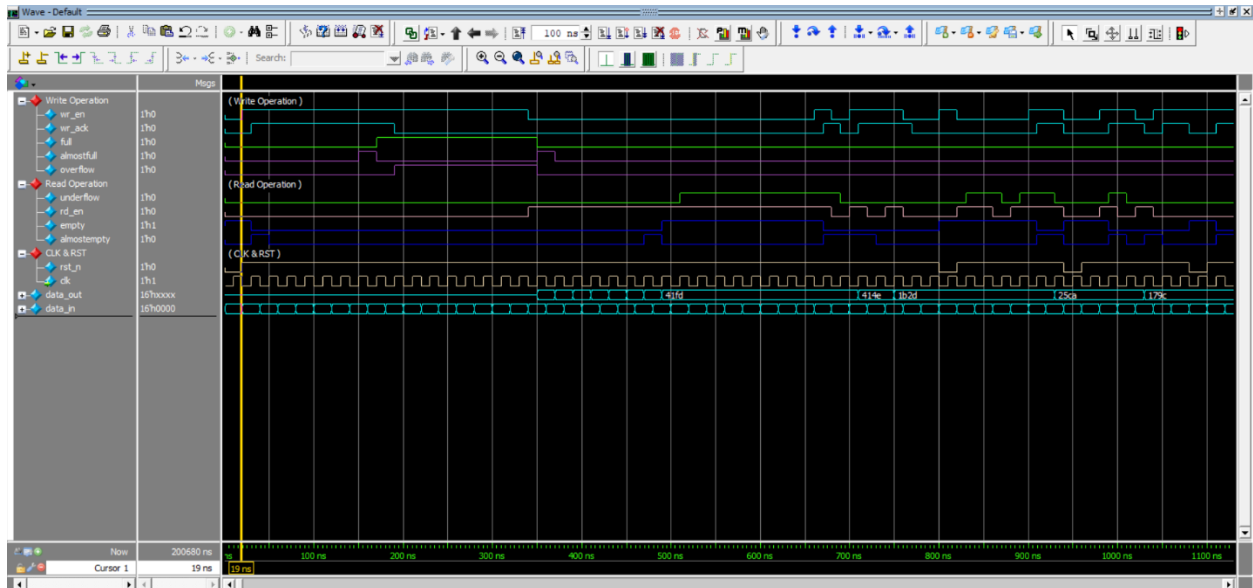
** Note: $finish : D:/Eng.Kareem/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
Time: 200680 ns Iteration: 61 Instance: /FIFO_TOP

# 1
# Break in Task uvm_pkg/uvm_root::run_test at D:/Eng.Kareem/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
V$SIM 10>
```

Questa Sim snippets to for each UVM sequence

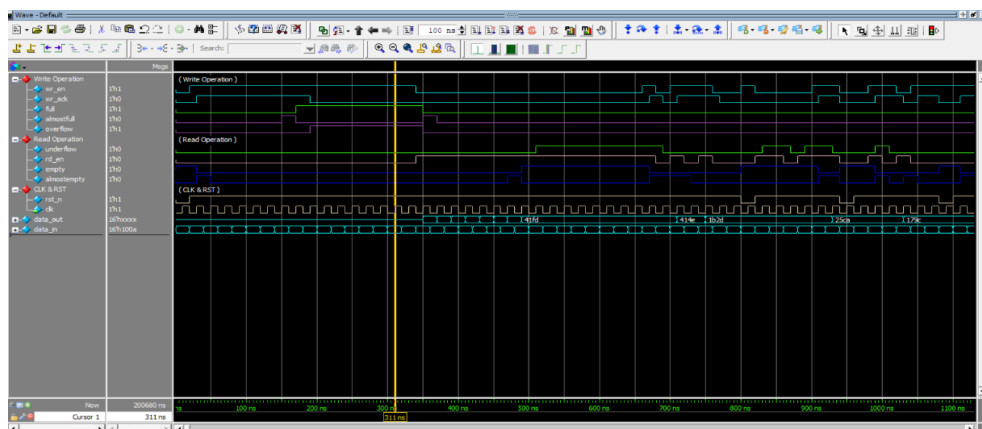
Sequence 1:

```
# UVM_INFO 10.FIFO_test_pkg.sv(51) @ 0: uvm_test_top [run_phase] Reset Asserted
# UVM_INFO 10.FIFO_test_pkg.sv(54) @ 20: uvm_test_top [run_phase] Reset Deasserted
```



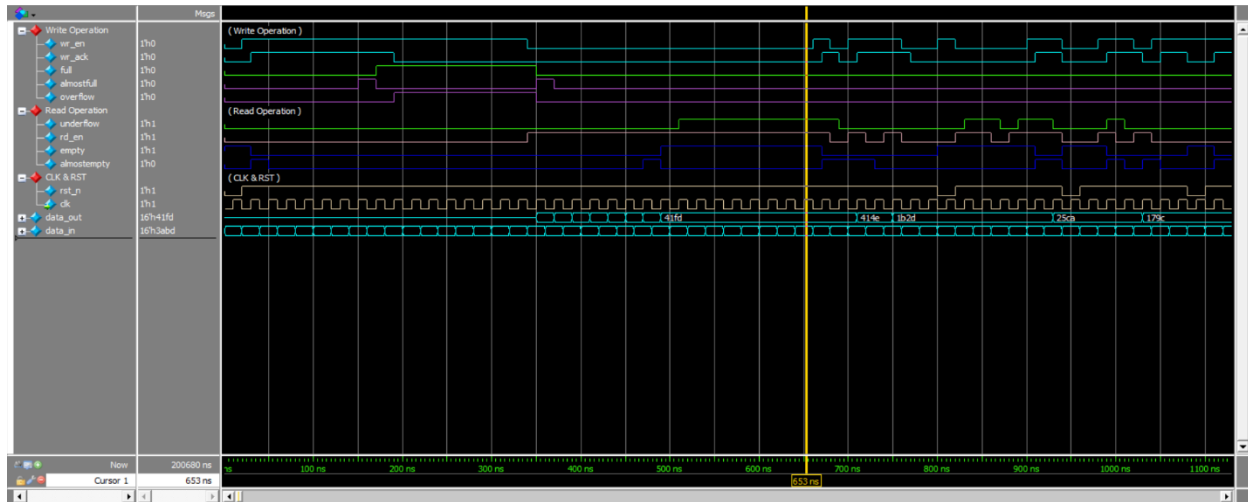
Sequence 2:

```
# UVM_INFO 10.FIFO_test_pkg.sv(57) @ 20: uvm_test_top [run_phase] Stimulus Generation write sequence 2 Started
# UVM_INFO 3.write_sequence_pkg.sv(21) @ 20: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 20: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 40: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 60: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 80: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 100: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 120: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 140: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 160: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 180: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 200: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 220: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 240: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 260: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 280: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 300: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(23) @ 320: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is started
# UVM_INFO 3.write_sequence_pkg.sv(35) @ 340: uvm_test_top.env.agt.sqr@wr_seq [body] 02 :: fifo_write_seq is ENDED
# UVM_INFO 10.FIFO_test_pkg.sv(60) @ 340: uvm_test_top [run_phase] Stimulus Generation write sequence 2 Ended
```



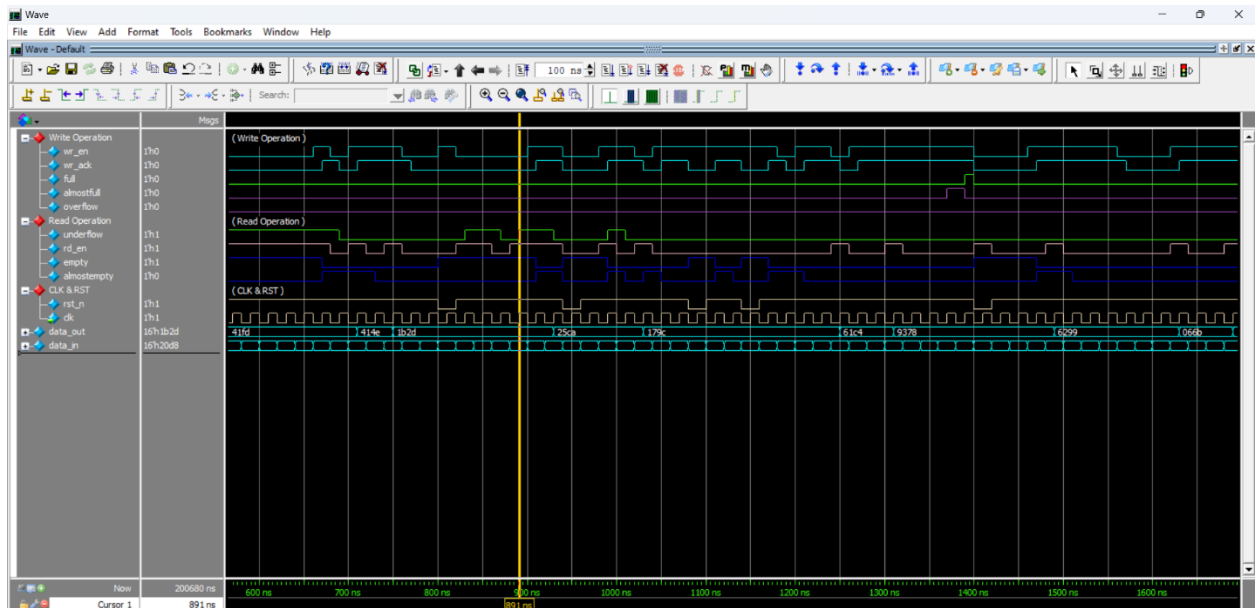
Sequence 3

```
# UVM_INFO 10.FIFO_test_pkg.sv(63) @ 340: uvm_test_top [run_phase] Stimulus Generation read sequence 3 Started
# UVM_INFO 3.reset_sequence_pkg.sv(52) @ 340: uvm_test_top.env.agt.sqr@@rd_seq [body] 03 :: fifo_read_seq is started
# UVM_INFO 3.reset_sequence_pkg.sv(64) @ 660: uvm_test_top.env.agt.sqr@@rd_seq [body] 03 :: fifo_read_seq is ENDED
# UVM_INFO 10.FIFO_test_pkg.sv(66) @ 660: uvm_test_top [run_phase] Stimulus Generation read sequence 3 Ended
```



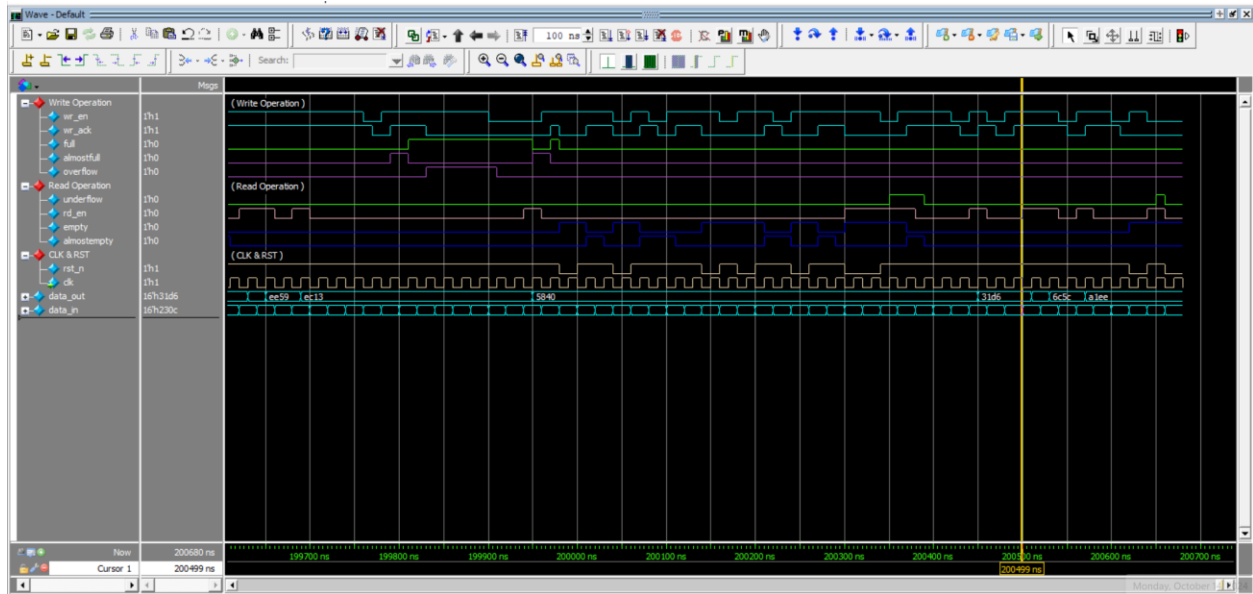
Sequence 4

```
# UVM_INFO 10.FIFO_test_pkg.sv(69) @ 660: uvm_test_top [run_phase] Stimulus Generation read after write sequence 4 Started
# UVM_INFO 3.reset_sequence_pkg.sv(82) @ 660: uvm_test_top.env.agt.sqr@@rd_wr_seq [body] 04 :: fifo_write_read_seq is started
# UVM_INFO 3.reset_sequence_pkg.sv(89) @ 200660: uvm_test_top.env.agt.sqr@@rd_wr_seq [body] 04 :: fifo_write_read_seq is ENDED
# UVM_INFO 10.FIFO_test_pkg.sv(72) @ 200660: uvm_test_top [run_phase] Stimulus Generation read after write sequence 4 Ended
```



Sequence 5

```
# UVM_INFO 10.FIFO_test_pkg.sv(75) @ 200660: uvm_test_top [run_phase] final :: Reset Asserted  
# UVM_INFO 10.FIFO_test_pkg.sv(78) @ 200680: uvm_test_top [run_phase] final :: Reset Deasserted
```



Codes

1. Top Module

```
1 import FIFO_test_pkg:: *;  
2 import uvm_pkg::* ;  
3 import FIFO_shared_pkg ::*;  
4  
5 `include "uvm_macros.svh"  
6  
7 module FIFO_TOP ();  
8  
9     bit clk;  
10  
11     //-----clock generation-----  
12     initial begin  
13         clk = 0 ;  
14         forever begin  
15             #10;  
16             clk = ~ clk;  
17         end  
18     end  
19  
20     //-----instantiating interface-----  
21     FIFO_if fifo_if (clk); // instantiate the interface  
22  
23     //----- DUT instantiation -----  
24     FIFO DUT (fifo_if); // Pass the instantiated interface to DUT  
25     bind FIFO fifo_sva fifo_sva_inst (fifo_if);  
26  
27  
28     //----- Test running-----  
29     initial begin  
30         uvm_config_db #(virtual FIFO_if)::set(null, "uvm_test_top", "fifo_if", fifo_if);  
31         run_test("fifo_test");  
32     end  
33  
34 endmodule  
35
```

2. Interface

```
1 interface FIFO_if(clk);
2
3     parameter FIFO_WIDTH = 16;
4     parameter FIFO_DEPTH = 8;
5
6     input bit clk;
7
8     logic [FIFO_WIDTH-1:0] data_in;
9     logic rst_n, wr_en, rd_en;
10    logic [FIFO_WIDTH-1:0] data_out;
11    logic wr_ack, overflow;
12    logic full, empty, almostfull, almostempty, underflow;
13
14    modport DUT (input data_in, wr_en, rd_en, clk, rst_n, output full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out);
15    modport TEST (output data_in, wr_en, rd_en, rst_n, input clk, full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out);
16    modport MONITOR (input data_in, wr_en, rd_en, clk, rst_n, full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out);
17
18 endinterface : FIFO_if
```

3. FIFO Test

```
1 package FIFO_test_pkg;
2     import FIFO_config_pkg::*;
3     import FIFO_env_pkg::*;
4     import reset_sequences_pkg::*;
5     import write_sequences_pkg::*;
6     import FIFO_shared_pkg::*;
7     import uvm_pkg::*;
8     `include "uvm_macros.svh"
9
10    class fifo_test extends uvm_test;
11        `uvm_component_utils(fifo_test)
12
13        fifo_env env;
14        virtual fifo_if fifo_test_vif;
15        fifo_config_obj fifo_config_obj_test;
16        fifo_write_seq wr_seq;
17        fifo_reset_seq reset_seq;
18        fifo_read_seq rd_seq;
19        fifo_write_read_seq rd_wr_seq;
20
21
22        function new(string name = "fifo_test", uvm_component parent = null);
23            super.new(name, parent);
24        endfunction
25
26        function void build_phase(uvm_phase phase);
27            super.build_phase(phase);
28            env = fifo_env::type_id::create("env", this);
29            fifo_config_obj_test = fifo_config_obj::type_id::create("fifo_config_obj_test");
30            reset_seq = fifo_reset_seq::type_id::create("reset_seq", this);
31            wr_seq = fifo_write_seq::type_id::create("wr_seq", this);
32            rd_seq = fifo_read_seq::type_id::create("rd_seq", this);
33            rd_wr_seq = fifo_write_read_seq::type_id::create("rd_wr_seq", this);
34
35
36            if (!uvm_config_db#(virtual FIFO_if)::get(this, "", "fifo_if", fifo_config_obj_test.fifo_config_vif ))
37
38                `uvm_fatal("build_phase", "Test - unable to get the virtual interface of fifo from the uvm_config_db")
39
40            uvm_config_db#(fifo_config_obj)::set(this, "", "CFG", fifo_config_obj_test);
41
42        endfunction
43    endclass
```

```

43
44
45     task run_phase(uvm_phase phase);
46         super.run_phase(phase);
47
48         phase.raise_objection(this);
49
50         // --- 1:: reset TEST 1
51         `uvm_info("run_phase","Reset Asserted", UVM_MEDIUM)
52         reset_seq.start(env.agt.sqr);
53
54         `uvm_info("run_phase","Reset Deasserted", UVM_MEDIUM)
55
56         //---2:: write TEST 2
57         `uvm_info("run_phase","Stimulus Generation write sequence 2 Started", UVM_MEDIUM)
58         wr_seq.start(env.agt.sqr);
59
60         `uvm_info("run_phase","Stimulus Generation write sequence 2 Ended", UVM_MEDIUM)
61
62         //---3:: read TEST 3
63         `uvm_info("run_phase","Stimulus Generation read sequence 3 Started", UVM_MEDIUM)
64         rd_seq.start(env.agt.sqr);
65
66         `uvm_info("run_phase","Stimulus Generation read sequence 3 Ended", UVM_MEDIUM)
67
68         //---4:: read TEST 4
69         `uvm_info("run_phase","Stimulus Generation read after write sequence 4 Started", UVM_MEDIUM)
70         rd_wr_seq.start(env.agt.sqr);
71
72         `uvm_info("run_phase","Stimulus Generation read after write sequence 4 Ended", UVM_MEDIUM)
73
74         //---5:: reset TEST 5
75         `uvm_info("run_phase","final :: Reset Asserted", UVM_MEDIUM)
76         reset_seq.start(env.agt.sqr);
77
78         `uvm_info("run_phase","final :: Reset Deasserted", UVM_MEDIUM)
79
80         phase.drop_objection(this);
81
82     endtask
83
84 endclass
85 endpackage
86

```

4. FIFO Environment

```
1 package FIFO_env_pkg;
2   import FIFO_driver_pkg::*;
3   import uvm_pkg::*;
4   import FIFO_scoreboard_pkg::*;
5   import FIFO_coverage_pkg::*;
6   import FIFO_agent_pkg::*;
7   import FIFO_shared_pkg::*;
8   `include "uvm_macros.svh"
9
10  class fifo_env extends uvm_env;
11    `uvm_component_utils(fifo_env)
12
13    fifo_scoreboard sb;
14    fifo_coverage cov;
15    fifo_agent agt;
16
17    function new(string name = "fifo_env", uvm_component parent = null);
18      super.new(name, parent);
19    endfunction
20
21    function void build_phase(uvm_phase phase);
22      super.build_phase(phase);
23      agt = fifo_agent::type_id::create("agt", this);
24      sb = fifo_scoreboard::type_id::create("sb", this);
25      cov = fifo_coverage::type_id::create("cov", this);
26    endfunction
27
28    function void connect_phase(uvm_phase phase);
29      agt.agt_port.connect(sb.sb_export);
30      agt.agt_port.connect(cov.cov_export);
31    endfunction
32
33  endclass
34 endpackage
35
```

5. FIFO Agent

```
1 package FIFO_agent_pkg;
2   import my_sequencer_pkg::*;
3   import FIFO_monitor_pkg::*;
4   import FIFO_driver_pkg::*;
5   import FIFO_config_pkg::*;
6   import FIFO_seq_item_pkg::*;
7
8
9   import uvm_pkg::*;
10  `include "uvm_macros.svh"
11
12  class fifo_agent extends uvm_agent;
13    `uvm_component_utils(fifo_agent)
14
15    MySequencer sqr;
16    fifo_driver drv;
17    fifo_monitor mon;
18    fifo_config_obj cfg;
19    uvm_analysis_port #(fifo_seq_item) agt_port;
20
21    function new(string name = "fifo_agent", uvm_component parent = null);
22      super.new(name, parent);
23    endfunction
24
25
26    function void build_phase(uvm_phase phase);
27      sqr = MySequencer::type_id::create("sqr", this);
28      mon = fifo_monitor::type_id::create("mon", this);
29      drv = fifo_driver::type_id::create("drv", this);
30
31      agt_port = new("agt_port", this);
32
33      if (!uvm_config_db #(fifo_config_obj)::get(this, "", "CFG", cfg)) begin
34        `uvm_fatal("build_phase", "Agent- Unable to get Configuration object :( ")
35      end
36    endfunction
37
38
39    function void connect_phase(uvm_phase phase);
40      super.connect_phase(phase);
41      drv.fifo_vif = cfg.fifo_config_vif;
42      mon.if_vif_mon = cfg.fifo_config_vif;
43
44      drv.seq_item_port.connect(sqr.seq_item_export);
45      mon.mon_port.connect(agt_port);
46
47    endfunction
48
49  endclass : fifo_agent
50
51
```


6. FIFO Monitor

```
1 package FIFO_monitor_pkg;
2   import FIFO_seq_item_pkg::*;
3   import FIFO_shared_pkg::*;
4
5   import uvm_pkg::*;
6   `include "uvm_macros.svh"
7
8   class fifo_monitor extends uvm_monitor;
9     `uvm_component_utils(fifo_monitor)
10
11     fifo_seq_item rsp_seq_item;
12     virtual FIFO_if if_vif_mon;
13     uvm_analysis_port #(fifo_seq_item) mon_port;
14
15     function new(string name = "fifo_monitor", uvm_component parent = null);
16       super.new(name, parent);
17
18     endfunction
19
20     function void build_phase(uvm_phase phase);
21       super.build_phase(phase);
22       mon_port = new("mon_port", this);
23
24     endfunction
25
26
27     task run_phase(uvm_phase phase);
28       super.run_phase(phase);
29       forever begin
30         rsp_seq_item = fifo_seq_item::type_id::create("rsp_seq_item");
31         @(negedge if_vif_mon.clk);
32
33         //inputs
34         rsp_seq_item.rst_n    = if_vif_mon.rst_n;
35         rsp_seq_item.data_in  = if_vif_mon.data_in;
36         rsp_seq_item.wr_en    = if_vif_mon.wr_en;
37         rsp_seq_item.rd_en    = if_vif_mon.rd_en;
38         rsp_seq_item.wr_ack   = if_vif_mon.wr_ack;
39
40         //inputs
41         rsp_seq_item.data_out  = if_vif_mon.data_out;
42         rsp_seq_item.overflow  = if_vif_mon.overflow;
43         rsp_seq_item.underflow = if_vif_mon.underflow;
44         rsp_seq_item.almostfull = if_vif_mon.almostfull;
45         rsp_seq_item.almostempty = if_vif_mon.almostempty;
46         rsp_seq_item.full      = if_vif_mon.full;
47         rsp_seq_item.empty     = if_vif_mon.empty;
48
49
50         mon_port.write(rsp_seq_item);
51       end
52
53       `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
54
55     endtask
56
57   endclass : fifo_monitor
58
59
60
61 endpackage : FIFO_monitor_pkg
```

7. FIFO Driver

```
1 package FIFO_driver_pkg;
2   import FIFO_shared_pkg::*;
3   import my_sequencer_pkg::*;
4   import FIFO_seq_item_pkg::*;
5
6   import uvm_pkg::*;
7   `include "uvm_macros.svh"
8
9   class fifo_driver extends uvm_driver #(fifo_seq_item);
10     `uvm_component_utils(fifo_driver)
11
12     virtual FIFO_if fifo_vif;
13     fifo_seq_item stim_seq_item;
14
15     function new(string name = "fifo_driver", uvm_component parent = null);
16       super.new(name, parent);
17
18     endfunction
19     function void build_phase (uvm_phase phase);
20       super.build_phase(phase);
21     endfunction
22
23     task run_phase(uvm_phase phase);
24       super.run_phase(phase);
25
26       forever begin
27         stim_seq_item = fifo_seq_item::type_id::create("stim_seq_item");
28         seq_item_port.get_next_item(stim_seq_item);
29
30         fifo_vif.data_in = stim_seq_item.data_in;
31         fifo_vif.wr_en   = stim_seq_item.wr_en;
32         fifo_vif.rd_en   = stim_seq_item.rd_en;
33
34         fifo_vif.rst_n   = stim_seq_item.rst_n;
35         @(negedge fifo_vif.clk);
36
37         seq_item_port.item_done();
38         `uvm_info("run_phase",stim_seq_item.convert2string_stimulus(), UVM_HIGH)
39       end
40     endtask
41
42   endclass : fifo_driver
43
44 endpackage
```

8. Configuration Object

```
1 package FIFO_config_pkg;
2   import uvm_pkg::* ;
3   `include "uvm_macros.svh"
4
5   //configuration object to hold the virtual interface from the test to the driver
6   class fifo_config_obj extends uvm_object;
7     `uvm_object_utils(fifo_config_obj)
8
9     virtual FIFO_if fifo_config_vif; //Inside this UVM object, we declare a virtual interface named fifo_config_vif.
10
11     function new(string name = "fifo_config_obj");
12       super.new(name);
13     endfunction
14
15   endclass
16 endpackage
```

9. Sequence Item

```
1 package FIFO_seq_item_pkg;
2   import FIFO_shared_pkg::*;
3   import uvm_pkg::*;
4   `include "uvm_macros.svh"
5
6   class fifo_seq_item extends uvm_sequence_item;
7     `uvm_object_utils(fifo_seq_item)
8
9     parameter FIFO_WIDTH = 16;
10    parameter FIFO_DEPTH = 8;
11
12    //Define items and add rand modifiers
13    bit clk;
14
15    rand logic [FIFO_WIDTH-1:0] data_in;
16    rand logic rst_n, wr_en, rd_en;
17    logic [FIFO_WIDTH-1:0] data_out;
18    logic wr_ack, overflow;
19    logic full, empty, almostfull, almostempty, underflow;
20
21    //constructor
22    function new(string name = "fifo_seq_item");
23      super.new(name);
24    endfunction : new
25
26
27    //functions convert to string --> Both inputs and Outputs
28    function string convert2string();
29      return $formatf("%s :: data_in = %0b, rst_n = %0b, wr_en = %0b, rd_en = %0b, data_out = %0b, wr_ack = %0b, overflow = %0b, full = %0b, empty = %0b, almostfull = %0b, almostempty = %0b, underflow = %0b",
30        super.convert2string(), data_in, rst_n, wr_en, rd_en, data_out, wr_ack, overflow, full, empty, almostfull, almostempty, underflow);
31    endfunction
32
33
34    //Add convert2string_stimulus function --> for Inputs Only
35    function string convert2string_stimulus();
36      return $formatf("data_in = %0b, rst_n = %0b, wr_en = %0b, rd_en = %0b ",
37        data_in, rst_n, wr_en, rd_en);
38    endfunction
39
40
41 endclass
```

```
41
42 //-----CONSTRAINTS-----
43 int RD_EN_ON_DIST = 30;
44 int WR_EN_ON_DIST = 70;
45
46
47 constraint rst_n_c {
48   rst_n_dist {0 := 10, 1 := 90};
49 }
50
51 constraint wr_en_c {
52   wr_en_dist {1 := WR_EN_ON_DIST, 0 := 100-WR_EN_ON_DIST};
53 }
54
55 constraint rd_en_c {
56   rd_en_dist {1 := RD_EN_ON_DIST, 0 := 100-RD_EN_ON_DIST};
57 }
58
59 endclass : fifo_seq_item
60
61
62 endpackage : FIFO_seq_item_pkg
```

10. FIFO Sequences Package

```
8
9 ▼ // -----Sequence 1 : RST -----
10
11 ▼ class fifo_reset_seq extends uvm_sequence #(fifo_seq_item);
12     `uvm_object_utils(fifo_reset_seq)
13     fifo_seq_item seq_item;
14
15 ▼     function new(string name = "fifo_reset_seq");
16         super.new(name);
17
18     endfunction
19
20 ▼     task body;
21         seq_item = fifo_seq_item::type_id::create("seq_item");
22 ▼         start_item(seq_item);
23
24             seq_item.rst_n = 0; // activate the active low rst_n
25             seq_item.data_in = 0; // to reset the whole data_in bus (of size FIFO_WIDTH)
26             seq_item.wr_en = 0;
27             seq_item.rd_en = 0;
28
29             finish_item(seq_item);
30
31     endtask : body
32 endclass : fifo_reset_seq
33
```

```
33
34 // -----Sequence 2 : write_only -----
35
36 class fifo_write_seq extends uvm_sequence #(fifo_seq_item);
37     `uvm_object_utils(fifo_write_seq)
38     fifo_seq_item seq_item_wr;
39
40     function new(string name = "fifo_write_seq");
41         super.new(name);
42
43     endfunction
44
45     task body;
46         `uvm_info("body","02 :: fifo_write_seq is started", UVM_MEDIUM);
47         repeat(FIFO_WIDTH)begin
48             `uvm_info("body","02 :: fifo_write_seq is started", UVM_MEDIUM);
49             seq_item_wr = fifo_seq_item::type_id::create("seq_item_wr");
50             start_item(seq_item_wr);
51
52                 assert(seq_item_wr.randomize());
53                 seq_item_wr.wr_en = 1;
54                 seq_item_wr.rd_en = 0;
55                 seq_item_wr.rst_n = 1;
56
57                 finish_item(seq_item_wr);
58
59             end
60             `uvm_info("body","02 :: fifo_write_seq is ENDED", UVM_MEDIUM);
61
62     endtask : body
63
64
```

```

38
39 // -----Sequence 3 : Read_only -----
40
41 class fifo_read_seq extends uvm_sequence #(fifo_seq_item);
42     `uvm_object_utils(fifo_read_seq)
43     fifo_seq_item seq_item_rd;
44
45     function new(string name = "fifo_read_seq");
46         super.new(name);
47
48     endfunction
49
50
51     task body;
52         `uvm_info("body","03 :: fifo_read_seq is started", UVM_MEDIUM);
53         repeat(FIFO_WIDTH)begin
54             seq_item_rd = fifo_seq_item::type_id::create("seq_item_rd");
55             start_item(seq_item_rd);
56
57             assert(seq_item_rd.randomize());
58             seq_item_rd.rd_en = 1;
59             seq_item_rd.wr_en = 0;
60             seq_item_rd.rst_n = 1;
61
62             finish_item(seq_item_rd);
63         end
64         `uvm_info("body","03 :: fifo_read_seq is ENDED", UVM_MEDIUM);
65
66     endtask : body
67
68 endclass : fifo_read_seq
69

```

```

0 // -----Sequence 4 : write_read_sequence -----
1
2 class fifo_write_read_seq extends uvm_sequence #(fifo_seq_item);
3     `uvm_object_utils(fifo_write_read_seq)
4     fifo_seq_item seq_item_wr_rd;
5
6     function new(string name = "fifo_write_read_seq");
7         super.new(name);
8
9     endfunction
10
11     task body;
12         `uvm_info("body","04 :: fifo_write_read_seq is started", UVM_MEDIUM);
13         repeat(10000)begin
14             seq_item_wr_rd = fifo_seq_item::type_id::create("seq_item_wr_rd");
15             start_item(seq_item_wr_rd);
16             assert(seq_item_wr_rd.randomize());
17             finish_item(seq_item_wr_rd);
18         end
19         `uvm_info("body","04 :: fifo_write_read_seq is ENDED", UVM_MEDIUM);
20
21     endtask : body
22
23 endclass : fifo_write_read_seq
24

```

11. FIFO Sequencer

```
1 package my_sequencer_pkg;
2     import FIFO_seq_item_pkg::*;
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5
6     class MySequencer extends uvm_sequencer #(fifo_seq_item);
7         `uvm_component_utils(MySequencer);
8
9         function new(string name = "MySequencer", uvm_component parent = null);
10             super.new(name, parent);
11         endfunction
12
13     endclass
14
15 endpackage
```

12. FIFO Scoreboard

```
1 package FIFO_scoreboard_pkg;
2   import FIFO_seq_item_pkg::*;
3   import FIFO_shared_pkg::*;
4   import uvm_pkg::*;
5   `include "uvm_macros.svh"
6
7   class fifo_scoreboard extends uvm_scoreboard;
8     `uvm_component_utils(fifo_scoreboard)
9
10    uvm_analysis_port #(fifo_seq_item) sb_export;
11    uvm_tlm_analysis_fifo #(fifo_seq_item) sb_fifo_port;
12    fifo_seq_item item;
13
14    // Declare variables to hold ref_model values
15    logic [FIFO_WIDTH-1:0] data_out_ref;
16    static int correct_count =0;
17    static int wrong_count =0;
18    parameter FIFO_WIDTH = 16;
19    logic wr_ack_ref, overflow_ref;
20    logic full_ref, empty_ref, almostfull_ref, almostempty_ref, underflow_ref;
21    static integer error_count=0 , right_count=0 , size ;
22    logic [FIFO_WIDTH-1:0] data_mem [$] ;
23    bit next_write,next_read;
24
25    // Error and correct counts
26
27    function new(string name = "fifo_scoreboard", uvm_component parent = null);
28      super.new(name, parent);
29    endfunction
30
31
32    function void build_phase(uvm_phase phase);
33      super.build_phase(phase);
34      sb_export = new("sb_export", this);
35      sb_fifo_port = new("sb_fifo_port", this);
36
37    endfunction
38
39
40    function void connect_phase(uvm_phase phase);
41      super.connect_phase(phase);
42      sb_export.connect(sb_fifo_port.analysis_export);
43
44    endfunction
45
```

```

46 // check the FIFO output against the reference model
47 task run_phase (ovm_phase phase);
48   super.run_phase(phase);
49   forever begin
50     sb_fifo_port.get(item);
51     check_data();
52   end
53 endtask
54
55 //-----Golden model-----
56
57 //Implement the reference model using a queue
58
59 task check_data ();
60   reference_model();
61   if (!item.rst_n) begin
62     if (item.wr_ack!=wr_ack_ref || item.overflow!=overflow_ref || item.full != full_ref || item.empty != empty_ref || item.almostfull != almostfull_ref || item.almostempty != almostempty_ref) begin
63       error_count=error_count+1;
64       $display("error at reset");
65       if (item.wr_ack!=wr_ack_ref) begin
66         $display("wr_ack=%0b , wr_ack_ref=%0b at %0t",item.wr_ack,wr_ack_ref,$time);
67       end
68       if (item.overflow!=overflow_ref) begin
69         $display("overflow=%0b , overflow_ref=%0b at %0t",item.overflow,overflow_ref,$time);
70       end
71       if (item.full!=full_ref) begin
72         $display("full=%0b , full_ref=%0b at %0t",item.full,full_ref,$time);
73       end
74       if (item.empty!=empty_ref) begin
75         $display("empty=%0b , empty_ref=%0b at %0t",item.empty,empty_ref,$time);
76       end
77       if (item.almostfull!=almostfull_ref) begin
78         $display("almostfull=%0b , almostfull_ref=%0b at %0t",item.almostfull,almostfull_ref,$time);
79       end
80       if (item.almostempty!=almostempty_ref) begin
81         $display("almostempty=%0b , almostempty_ref=%0b at %0t",item.almostempty,almostempty_ref,$time);
82       end
83     end
84   else begin
85     right_count = right_count+1;
86   end
87 end
88
89 else begin
90   if (item.wr_ack!=wr_ack_ref || item.overflow!=overflow_ref || item.data_out!=data_out_ref || item.full != full_ref || item.empty != empty_ref || item.almostfull != almostfull_ref || item.almostempty != almostempty_ref) begin
91     error_count=error_count+1;
92     $display("error at no reset");
93     if (item.wr_ack!=wr_ack_ref) begin
94       $display("wr_ack=%0b , wr_ack_ref=%0b , size=%0d at %0t",item.wr_ack,wr_ack_ref,data_mem.size(),$time);
95     end
96     if (item.overflow!=overflow_ref) begin
97       $display("overflow=%0b , overflow_ref=%0b at %0t",item.overflow,overflow_ref,$time);
98     end
99     if (item.data_out!=data_out_ref) begin
100       $display("dataout=%0d , dataout_ref=%0d , size=%0d at %0t",item.data_out,data_out_ref,data_mem.size(),$time);
101     end
102     if (item.full!=full_ref) begin
103       $display("full=%0b , full_ref=%0b at %0t",item.full,full_ref,$time);
104     end
105     if (item.empty!=empty_ref) begin
106       $display("empty=%0b , empty_ref=%0b , size=%0d at %0t",item.empty,empty_ref,data_mem.size(),$time);
107     end
108     if (item.almostfull!=almostfull_ref) begin
109       $display("almostfull=%0b , almostfull_ref=%0b at %0t",item.almostfull,almostfull_ref,$time);
110     end
111     if (item.almostempty!=almostempty_ref) begin
112       $display("almostempty=%0b , almostempty_ref=%0b , size=%0d at %0t",item.almostempty,almostempty_ref,data_mem.size(),$time);
113     end
114   end
115   right_count = right_count+1;
116 end
117 endtask
118
119 task reference_model();
120 if (!item.rst_n) begin
121   full_ref=0;
122   almostfull_ref=0;
123   empty_ref=1;
124   almostempty_ref=0;
125   overflow_ref=0;
126   underflow_ref=0;
127   data_mem.delete();
128   wr_ack_ref=0;
129   next_read=0;
130   next_write=0;
131 end
132 else
133   begin
134     size=data_mem.size();
135     // read
136     if (item.rd_en && size>0) begin
137       data_out_ref=data_mem.pop_front();
138     end
139
140     if (item.wr_en && size<8) begin
141       data_mem.push_back(item.data_in);
142       wr_ack_ref=1;
143     end
144   end
145 end

```



```

145         data_mem.push_back(item.data_in);
146         wr_ack_ref=1;
147     end
148     else
149     begin
150         wr_ack_ref=0;
151     end
152     if(item.wr_en&&next_write)
153     begin
154         overflow_ref=1;
155     end
156     else begin
157         overflow_ref=0;
158     end
159     if(item.rd_en&&next_read)
160     begin
161         underflow_ref=1;
162     end
163     else begin
164         underflow_ref=0;
165     end
166     if (data_mem.size==7) begin
167         almostfull_ref=1;
168     end
169     else begin
170         almostfull_ref=0;
171     end
172     if (data_mem.size==8) begin
173         full_ref=1;
174         next_write=1;
175     end
176     else begin
177         full_ref=0;
178         next_write=0;
179     end
180     if (data_mem.size==0) begin
181         empty_ref=1;
182         next_read=1;
183     end
184     else begin
185         empty_ref=0;
186         next_read=0;
187     end
188     if (data_mem.size==1) begin
189         almostempty_ref=1;
190     end
191     else begin
192         almostempty_ref=0;
193     end
194 end
195 endtask
196
197
198 //-----reporting-----

```

```

190         end
191     else begin
192         almostempty_ref=0;
193     end
194 end
195 endtask
196
197
198 //-----reporting-----
199 function void report_phase(uvm_phase phase);
200     super.report_phase(phase);
201     `uvm_info("report_phase", $sformatf("Total Successful transaction: %0d",right_count), UVM_MEDIUM);
202     `uvm_info("report_phase", $sformatf("Total Failed transaction: %0d",error_count), UVM_MEDIUM);
203 endfunction
204
205 endclass
206
207 endpackage

```

13. FIFO Coverage Collector

```

1 package FIFO_coverage_pkg;
2 import FIFO_seq_item_pkg::*;
3 import FIFO_shared_pkg::*;
4 import uvm_pkg::*;
5 `include "uvm_macros.svh"
6
7 class fifo_coverage extends uvm_component;
8     `uvm_component_utils(fifo_coverage)
9
10     uvm_analysis_export #(fifo_seq_item) cov_export;
11     uvm_tlm_analysis_fifo #(fifo_seq_item) cov_fifo;
12     fifo_seq_item seq_item_cov;
13
14     //-----Coverage group-----
15
16     covergroup cvr_gp();
17         wr_en_cp      : coverpoint seq_item_cov.wr_en;
18         rd_en_cp      : coverpoint seq_item_cov.rd_en;
19         wr_ack_cp      : coverpoint seq_item_cov.wr_ack;
20         overflow_cp     : coverpoint seq_item_cov.overflow;
21         full_cp        : coverpoint seq_item_cov.full;
22         empty_cp       : coverpoint seq_item_cov.empty;
23         almostfull_cp  : coverpoint seq_item_cov.almostfull;
24         almostempty_cp : coverpoint seq_item_cov.almostempty;
25         underflow_cp   : coverpoint seq_item_cov.underflow;
26
27
28         ack_cross: cross wr_en_cp, rd_en_cp, wr_ack_cp[
29             ignore_bins case 011 bin = binsof(wr_en_cp) intersect {0} && binsof(wr_ack_cp) intersect {1};
30             //should exclude case of wr_en_cp = 0 rd_en_cp = 1, wr_ack_cp = 1 for coverage "how could be the wr_ack=1, while wr_en not enabled!"
31             ignore_bins case 001 bin = binsof(wr_en_cp) intersect {0} && binsof(rd_en_cp) intersect {0} && binsof(wr_ack_cp) intersect {1};
32             //should exclude case of wr_en_cp = 0 rd_en_cp = 0, wr_ack_cp = 1 for coverage "how could be the wr_ack = 1, while nethier wr_en nor read is enabled!"
33         ]
34         full_cross : cross wr_en_cp, rd_en_cp, full_cp[
35             ignore_bins read_full_bin = binsof(rd_en_cp) intersect {1} && binsof(full_cp) intersect {1};
36             // As the full flag main function is when write enable on we check on if the fifo if full or not
37             // as it indicates that the FIFO is full. Write requests are ignored when the FIFO is full
38             // initiating a write when the FIFO is full is not destructive to the contents of the FIFO
39         ]
40
41         }
42         almostfull_cross : cross wr_en_cp, rd_en_cp, almostfull_cp;
43         empty_cross      : cross wr_en_cp, rd_en_cp, empty_cp;
44         almostempty_cross : cross wr_en_cp, rd_en_cp, almostempty_cp;
45         overflow_cross    : cross wr_en_cp, rd_en_cp, overflow_cp[
46             ignore_bins write_with_overflow_bin = binsof(wr_en_cp) intersect {0} && binsof(overflow_cp) intersect {1};
47             // As the overflow flag main function is when write enable on indicates that a write request (wr_en) was rejected
48             // because the FIFO is full. Overflowing the FIFO is not destructive to the contents of the FIFO
49         ]
50         underflow_cross : cross wr_en_cp, rd_en_cp, underflow_cp[
51             ignore_bins rd0und1 = binsof(rd_en_cp) intersect {0} && binsof(underflow_cp) intersect {1};
52             // As the Underflow flag main function is Indicates that the read request(rd_en) was rejected
53             //because the FIFO is empty. Under flowing the FIFO is not destructive to the FIFO
54         ]
55     ]
56
57     endgroup
58
59     //Constructor
60     function new(string name = "fifo_coverage", uvm_component parent = null);
61         super.new(name,parent);
62         cvr_gp = new();
63     endfunction
64
65     //build
66     function void build_phase(uvm_phase phase);
67         super.build_phase(phase);
68         cov_export = new("cov_export",this);
69         cov_fifo = new("cov_fifo",this);
70     endfunction
71
72
73     //connect
74     function void connect_phase(uvm_phase phase);
75         super.connect_phase(phase);
76         cov_export.connect(cov_fifo.analysis_export);
77     endfunction
78
79
80     //run
81     task run_phase(uvm_phase phase);
82         super.run_phase(phase);
83         forever begin
84             cov_fifo.get(seq_item_cov);
85             cvr_gp.sample();
86         end
87     endtask
88
89
90
91     endclass
92
93 endpackage

```

14. Full Edited Design

```
1 ////////////////////////////////////////////////////////////////////
2 // Author: Kareem Waseem
3 // Course: Digital Verification using SV & UVM
4 //
5 // Description: FIFO Design
6 //
7 ////////////////////////////////////////////////////////////////////
8 module FIFO(FIFO_if.DUT fifo_if);
9
10     parameter FIFO_WIDTH = 16;
11     parameter FIFO_DEPTH = 8;
12     /*
13     input [FIFO_WIDTH-1:0] data_in;
14     input clk, rst_n, wr_en, rd_en;
15     output reg [FIFO_WIDTH-1:0] data_out;
16     output reg wr_ack, overflow;
17     output reg full, empty, almostfull, almostempty, underflow;
18     */
19     logic [fifo_if.FIFO_WIDTH-1:0] data_in;
20     logic clk, rst_n, wr_en, rd_en;
21     logic [fifo_if.FIFO_WIDTH-1:0] data_out;
22     logic wr_ack, overflow;
23     logic full, empty, almostfull, almostempty, underflow;
24
25
26     assign clk = fifo_if.clk;
27     assign data_in = fifo_if.data_in;
28     assign rst_n = fifo_if.rst_n;
29     assign wr_en = fifo_if.wr_en;
30     assign rd_en = fifo_if.rd_en;
31     assign fifo_if.data_out = data_out;
32     assign fifo_if.wr_ack = wr_ack;
33     assign fifo_if.overflow = overflow;
34     assign fifo_if.full = full;
35     assign fifo_if.empty = empty;
36     assign fifo_if.almostfull = almostfull;
37     assign fifo_if.almostempty = almostempty;
38     assign fifo_if.underflow = underflow;
39     //-----wires-----
40
41     localparam max_fifo_addr = $clog2(FIFO_DEPTH);
42
43     reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
44
45     reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
46     reg [max_fifo_addr:0] count;
47
48     //-----Write Operation-----
49
50
51     //-----Write Operation-----
52
53     always @(posedge clk or negedge rst_n) begin
54         if (!rst_n) begin
55             wr_ptr <= 0;
56
57             //----- Bug 4 & 5: I added those lines for overflow & wr_ack bec reset resets all the signals related to write operation :D
58             overflow <= 0;
59             wr_ack <= 0;
60
61         end
62         else if (wr_en && count < FIFO_DEPTH) begin
63             mem[wr_ptr] <= data_in;
64             wr_ack <= 1;
65             wr_ptr <= wr_ptr + 1;
66             overflow <= 0;
67
68         end
69         else begin
70             wr_ack <= 0;
71             if (full & wr_en)
72                 overflow <= 1;
73             else
74                 overflow <= 0;
75
76         end
77     end
78
79     //-----Read Operation-----
```

```

73
74 //-----Read Operation-----
75
76 always @(posedge clk or negedge rst_n) begin
77     if (!rst_n) begin
78         rd_ptr <= 0;
79         underflow <= 0; //-----> Bug 2: i added this line because reset resets underflow
80     end
81     else if (rd_en && count != 0) begin
82         data_out <= mem[rd_ptr];
83         rd_ptr <= rd_ptr + 1;
84         underflow <= 0;
85     end
86     else begin //-----> Bug 3 : this else branch i added to handle the underflow signal in a sequential manner (so i use non-blocking)
87         if (empty && rd_en) begin
88             underflow <= 1;
89         end
90         else
91             underflow <= 0;
92     end
93 end
94
95 end
96
97
98 //-----Counter -----
99
100
101 always @(posedge clk or negedge rst_n) begin
102     if (!rst_n) begin
103         count <= 0;
104     end
105     else begin
106         if ((wr_en, rd_en) == 2'b10) && !full)
107             count <= count + 1;
108         else if ((wr_en, rd_en) == 2'b01) && !empty)
109             count <= count - 1;
110
111         //-----> Bug 6 : this else branch i added to handle the counter incrementation If a read and write enables were high and the FIFO was empty
112         else if ((wr_en, rd_en) == 2'b11) && empty)
113             count <= count + 1;
114
115         //-----> Bug 7 : this else branch i added to handle the counter decrementation If a read and write enables were high and the the FIFO was full.
116         else if ((wr_en, rd_en) == 2'b11) && full)
117             count <= count - 1;
118     end
119 end
120
121 assign full = (count == FIFO_DEPTH)? 1 : 0;
122 assign empty = (count == 0)? 1 : 0;
123
124 //-----> Bug 1: underflow should be sequential ---should removed--> assign underflow = (empty && rd_en)? 1 : 0;
125 assign almostfull = (count == FIFO_DEPTH-1)? 1 : 0; // Bug 8 : fifo depth -2 --> fifo depth -1
126 assign almostempty = (count == 1)? 1 : 0;
127
128
129 endmodule
130

```

Thank
you!

